

Summary: NumPy-I

This report provides a comprehensive summary of the foundational concepts of NumPy, as detailed in the sources. You can copy this content directly into a Word document to create a permanent study reference.

1. Introduction to NumPy

NumPy, short for **Numerical Python**, is the fundamental library for scientific computing within the Python ecosystem. It is primarily used to provide support for large, multi-dimensional arrays and matrices, alongside a vast collection of mathematical functions to operate on these structures.

- **Importance:** It is essential for efficient numerical computations and serves as the foundation for many other scientific libraries. It also includes statistical functions (mean, median, standard deviation, and variance) to help understand data distributions.
 - **Installation:** Use `conda install numpy` or `pip install numpy`.
 - **Import Convention:** It is a widely adopted convention to use `import numpy as np` to enhance code readability.
-

2. Understanding the Array (`ndarray`)

An **array** is a data type used to store multiple values using a single identifier. While Python has a built-in array structure, the NumPy version—officially called an **ndarray**—is more versatile and efficient.

Key Characteristics:

- **Homogeneity:** Every element in the array must be of the same data type.
 - **Contiguous Memory:** Elements are stored next to each other in memory, making operations significantly faster than standard Python lists.
 - **Indexing:** Elements are referenced by their unique integral position (index), starting from **0**.
-

3. Array Creation and Data Types

NumPy arrays can be created from Python lists or tuples using the `np.array()` function.

- **1-D Array:** A single row of elements.
- **2-D Array (Matrix):** Elements organised into **rows (Axis 0)** and **columns (Axis 1)**.
- **3-D Array:** A collection of 2-D arrays stacked along a third axis. **Axis 0** represents matrices, **Axis 1** represents rows, and **Axis 2** represents columns.

Example: Creating a 2-D Array

```
import numpy as np
# Creating a 3x3 matrix
matrix = np.array([, , ])
```

Data Types (dtype): You can specify the data type (e.g., `float`) during creation using the `dtype` argument. If mixed types are provided, NumPy will **upcast** the elements to a common type, such as Unicode strings, to maintain homogeneity. To keep original mixed types, you must specify `dtype=object`.

4. Essential Array Attributes

The sources identify several key attributes used to inspect the structure of an `ndarray`:

1. **ndim:** The number of dimensions (axes).
 2. **shape:** A tuple representing the size of the array along each dimension (e.g., `(rows, columns)`).
 3. **size:** The total number of elements, calculated as the product of the `shape`.
 4. **dtype:** The data type of the internal elements.
 5. **itemsize:** The memory size (in bytes) of each individual element.
 6. **data:** A direct buffer to the memory location of the array's data.
-

5. Indexing and Slicing

Integer and Boolean Indexing

- **Integer Indexing:** Access elements by position. For a 2-D array, use `[row, column]`; for 3-D, use `[matrix, row, column]`.
- **Boolean Indexing (Masking):** Use a True/False array (a mask) of the same shape to select elements where the mask is `True`. This typically results in a **flattened** (1-D) array.

Slicing

Slicing uses the `start:stop:step` syntax.

- **Multi-dimensional Slicing:** Separate slices for each dimension with commas (e.g., `arr[1:3, 1:3]`).
 - **Ellipsis (...):** Used to represent all remaining dimensions. For example, `arr[:, :, 0]` would select the first column across all matrices and rows.
-

6. Arithmetic Operations

NumPy performs **element-wise** operations, meaning the operation is applied to each corresponding pair of elements between two arrays. For these operations to work, the arrays **must have the same shape**.

Example: Element-wise Operations

```
a = np.array()
b = np.array()

# Addition:
print(a + b)

# Exponentiation:
print(a ** 2)
```

Supported operations include addition, subtraction, multiplication, division, modulus, exponentiation, and floor division.

Analogy for Understanding Arrays: Think of a standard Python list like a **shopping bag** where you can throw in different items (milk, eggs, a magazine). It is flexible but hard to organise. A NumPy array is like a **branded egg carton**; every slot is the exact same size, designed for the exact same type of item, and arranged in a perfect grid. This rigid structure is what allows you to process thousands of "eggs" much faster than searching through a messy bag.